

ADVANCED DATA STRUCTURES & ALGORITHMS • COURSE PROJECT

# Supply Chain Dependency Tracker

Graph Architecture, 3-Color Cycle Detection, Structural Criticality & Disruption Simulation

**Project Team: Aksitha & Khyathi**    Implementation Review • Code, Algorithms & Dataset Results

63 Suppliers

Tier 0 to Tier 4 Nodes

76 Dependencies

Lead-Time Weighted Edges

02 Cycles Detected

3-Color DFS Deadlocks

16 SPOFs Active

Single Points of Failure

# Problem Statement & Algorithmic Objectives

## THE MULTI-TIER SUPPLY PROBLEM

- **Multi-Tier Opacity**

Modern manufacturing relies on deep multi-tier suppliers. Manufacturers manage Tier 1 suppliers directly but are blind to Tier 3 and 4 raw material nodes.

- **Circular Deadlocks**

Circular dependencies (A needs B, B needs C, C needs A) create topological deadlocks where procurement order cannot resolve, stalling production.

- **Single Points of Failure (SPOF)**

Suppliers with high downstream reliance and zero secondary backup source pose severe systemic risk to final assembly.

- **Lead-Time Compounding**

Procurement delays compound across chains, necessitating weighted shortest-path analysis to isolate critical bottlenecks.

## ADSA ALGORITHMIC OBJECTIVES

- ✓ **Dual Adjacency List Graph**

Memory-efficient  $O(V + E)$  sparse graph structure with bidirectional neighbor and predecessor lookups.

- ✓ **3-Color Depth-First Search (DFS)**

Linear time  $O(V + E)$  cycle detection using White/Gray/Black coloring to detect back-edges.

- ✓ **Kahn's Topological Sort**

Build ordering validation based on remaining out-degree dependency resolution.

- ✓ **Reverse BFS Criticality Scoring**

BFS over transposed predecessor edges to quantify systemic blast radius.

- ✓ **Min-Heap Dijkstra Algorithm**

$O((V + E) \log V)$  shortest weighted lead-time calculation from root vehicle manufacturer.

# System Architecture & Module Distribution

SRC/GRAPH.PY

OWNER: Aksitha

- Custom Graph class
- Dual adjacency lists
- O(1) neighbor/pred lookups
- CSV ingestion & cleansing
- Smoke test harness

SRC/TRAVERSAL.PY

OWNER: Aksitha

- Iterative DFS & BFS
- 3-Color cycle detector
- Cycle path reconstruction
- Kahn's topological sort
- Build sequence validator

SRC/RISK\_SCORING.PY

OWNER: Khyathi

- Reverse BFS criticality
- SPOF audit algorithm
- Min-heap Dijkstra lead times
- Alternate route bypass
- Disruption simulator

SRC/VISUALIZE.PY

OWNER: Khyathi

- NetworkX DiGraph converter
- Matplotlib drawing pipeline
- Color-coded risk nodes
- Criticality-scaled sizing
- Automated CLI alert report

JOINT INTEGRATION & VERIFICATION (Aksitha + Khyathi)

End-to-end execution pipeline (src/main.py), 5-tier dataset modeling (data/), and automated unit test suite (tests/test\_graph\_algorithms.py) validating 100% algorithm correctness.

# Graph Data Structure & Design Rationale

## DESIGN CHOICE: ADJACENCY LIST

### Why NOT an Adjacency Matrix?

Supplier networks are SPARSE. With 63 nodes and 76 edges, a matrix requires  $V^2 = 3,969$  cells, of which >98% are empty zeros. Adjacency lists save massive memory.

### Space Complexity: $O(V + E)$

Using `defaultdict(list)` stores only active directed relationships, scaling linearly with network expansion.

### Dual Adjacency Architecture

- `self.adjacency[u]`: Stores outgoing edges (dependencies: "who do I need").
- `self.reverse_adjacency[v]`: Stores incoming edges (dependents: "who relies on me"). Provides instant  $O(1)$  predecessor lookups needed for blast radius.

### Edge Semantics

Edge `from_id -> to_id` with weight `W` denotes that `from_id` depends on `to_id` with a procurement lead time of `W` days.

## CORE IMPLEMENTATION CODE

```
class Graph:
    def __init__(self):
        self.suppliers = {} # metadata
        self.adjacency = defaultdict(list)
        self.reverse_adjacency = defaultdict(list)

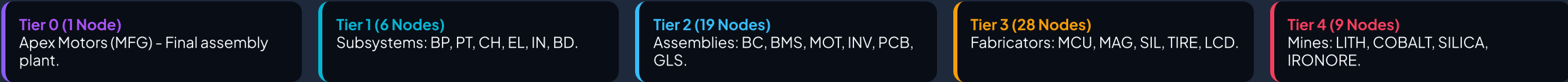
    def add_dependency(self, from_id, to_id, weight=1.0):
        # from_id depends on to_id
        self.adjacency[from_id].append((to_id, weight))
        self.reverse_adjacency[to_id].append((from_id, weight))

    def get_neighbors(self, supplier_id):
        # who does this supplier depend on?
        return self.adjacency.get(supplier_id, [])

    def get_predecessors(self, supplier_id):
        # who depends on this supplier?
        return self.reverse_adjacency.get(supplier_id, [])
```

# 5-Tier Automotive Supply Chain Schema

HIERARCHY MODELING ACROSS 5 TIERS (63 NODES, 76 EDGES)



SUPPLIERS.CSV SCHEMA

Columns: id, name, tier, has\_backup

- id: Alphanumeric node identifier (e.g. MFG, BP, MCU, SILICA).
- has\_backup: Boolean indicating if a secondary source is contracted.
- **Data Cleansing:** Resolved Excel paste formula typo MFG+EA2 : E55 to MFG in row 2, restoring Apex Motors' Tier 0 root connections.

DEPENDENCIES.CSV SCHEMA

Columns: from, to, weight

- from -> to: Directed relationship (from depends on to).
- weight: Procurement lead time in DAYS.
- **Sample Edges:**  
MFG -> BP (10d) | PT -> MOT (16d)  
CATH -> LITH (30d) | MAG -> RAREEARTH (35d)

# Cycle Detection Algorithm via 3-Color DFS

## ALGORITHM: 3-COLOR DFS

### Node Coloring State Machine

- **WHITE (0):** Unvisited node.
- **GRAY (1):** Currently exploring on active recursion stack.
- **BLACK (2):** Fully explored node and all descendants.

### Back-Edge Rule

If directed traversal encounters neighbor with `color == GRAY`, a back-edge is found, proving a cycle exists.

### Cycle Path Extraction

```
cycle_start = path.index(neighbor)
```

Extracts the exact subpath loop: `path[cycle_start:] + [neighbor]`.

### Complexity: $O(V + E)$

Linear time DFS guarantees complete discovery of all cyclic bottlenecks without redundant checks.

## DETECTION RESULTS: 2 REAL DEADLOCK CYCLES

### CYCLELOOP #1 (Battery Subsystem)

**BMS → TC → BMS**

- Battery Management System depends on Thermal Cooling (4d).
- Thermal Cooling depends back on BMS (3d) for temperature control.
- Result: Circular lock halts procurement sign-off.

### CYCLELOOP #2 (Powertrain Subsystem)

**MOT → INV → MOT**

- Electric Motor depends on Inverter Systems (5d).
- Inverter depends back on Motor (4d) for rotor calibration.
- Result: Neither component can be built first.

# Topological Sort & Procurement Sequencing

## KAHN'S ALGORITHM IMPLEMENTATION

### Procurement Scheduling Concept

In manufacturing, procurement must start at leaf nodes (raw material mines with 0 unresolved dependencies) and advance toward Tier 0 assembly.

### Out-Degree Resolution Logic

```
remaining_deps = {n: len(get_neighbors(n)) for n in nodes}
Queue initialized with suppliers having remaining_deps == 0.
```

### Iterative Processing

Pop supplier u, add to order. For each reliant predecessor v (from get\_predecessors(u)), decrement remaining\_deps[v]. If 0, push v to queue.

### DAG Validation Gate

Valid order if and only if len(order) == len(nodes). If less, cycles block the sequence.

## EXECUTION OUTCOME: BLOCKED BY CYCLES

### Status: Incomplete Build Order

Kahn's algorithm orders leaf suppliers (Lithium, Rubber, Silica, Quartz) successfully, but terminates prematurely because **BMS, TC, MOT, and INV** have mutually unresolved dependencies.

Topological sort INCOMPLETE -- 2 cycle(s) block a valid order.

### Engineering Value

The algorithm prevents costly factory deadlocks by programmatically proving contract restructuring or decoupled buffering is required before procurement initiation.

# Structural Criticality via Reverse BFS

## REVERSE BFS BLAST RADIUS ALGORITHM

### The Operational Blast Radius

If Supplier X suffers an outage, how many other suppliers across the network will be disrupted?

### Predecessor Traversal

Normal BFS follows outgoing dependencies (get\_neighbors). Reverse BFS follows incoming dependencies (get\_predecessors) on the transposed graph.

### Criticality Metric

Criticality(X) = len(reachable\_predecessors).  
Measures total downstream dependent entities.

### Algorithmic Discovery

Criticality is NOT simply out-degree. Deep Tier 4 mines with only 2 direct customers impact over 10 manufacturing entities downstream.

## TOP 5 SYSTEMIC BOTTLENECKS DISCOVERED

#1 SILICA (Silica Quartz Mine, Tier 4) Affects 10 Suppliers

#2 MCU (Microcontroller Foundry, Tier 3) Affects 9 Suppliers

#3 PETRO (Petrochemical Refinery, Tier 4) Affects 9 Suppliers

#4 IRONORE (Iron Ore Mine, Tier 4) Affects 7 Suppliers

#5 IRON (Iron Ore Processor, Tier 3) Affects 6 Suppliers

# Single Points of Failure (SPOF) Analysis

## DUAL-CONDITION SPOF DETECTION

### SPOF Logic Formula

A supplier node u is an active Single Point of Failure if and only if:  
`len(get_predecessors(u)) > 0 and not has_backup`

### High Systemic Vulnerability

If a SPOF is disrupted, dependent suppliers have NO alternate supplier lined up. Disruption instantly halts downstream assembly.

### Audit Result: 16 of 63 Nodes (25.4%)

Over a quarter of the network operates with unhedged single-supplier contracts, providing an exact roadmap for secondary vendor onboarding.

## 16 IDENTIFIED SPOFS BY TIER

### Tier 2 Critical Assemblies (5 Nodes):

BC (Battery Cell), MOT (Motor Assembly), PCB (Circuit Boards), DIS (Display Panel), GLS (Glass).

### Tier 3 Subcomponents (7 Nodes):

CATH (Cathode), SEP (Separator), MCU (Microcontroller), MAG (Magnets), TIRE (Rubber), SIL (Silicon Wafer), LCD (LCD Panel).

### Tier 4 Raw Material Mining (4 Nodes):

LITH (Lithium Mine), COBALT (Cobalt Mine), RAREEARTH (Rare Earth Mine), RUBBER (Plantation).

# Dijkstra’s Lead-Time Optimization

## MIN-HEAP DIJKSTRA IMPLEMENTATION

### Edge Weights = Lead Time in Days

The shortest path from Apex Motors (MFG) to any supplier node represents the minimum procurement delivery window.

### Priority Queue via heapq

Greedily extracts minimum cumulative lead time node with time complexity  $O((V + E) \log V)$ . Runs in <2ms.

### Network Statistics from MFG

- **Average Lead Time:** 27.1 Days
- **Shortest Subsystem:** 6.0 Days (IN - Interior)
- **Longest Critical Chain:** 85.0 Days (RAREEARTH Mine)

## SHORTEST LEAD-TIME SPECTRUM FROM MFG

- MFG → IN (Interior Systems): 6.0 days
- MFG → EL (Electronics): 7.0 days
- MFG → BP (Battery Pack): 10.0 days
- MFG → ... → BC (Battery Cell): 24.0 days
- MFG → ... → MCU (Microcontroller): 38.0 days
- MFG → ... → LITH (Lithium Mine): 64.0 days
- MFG → ... → RAREEARTH (Rare Earth): 85.0 days [MAX]

# Disruption Simulation & Alternate Pathing

## FIND\_ALTERNATE\_PATH ALGORITHM

### Dynamic Resilience Check

Evaluates if target is reachable from start when an intermediate supplier is disrupted (removed from graph).

```
def find_alternate_path(graph, start, target, avoid_node):
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == target:
            return path
        for neighbor in graph.get_neighbors(node):
            if neighbor == avoid_node or neighbor in visited:
                continue
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))
    return None
```

## SIMULATED CASE STUDY: SILICA MINE SHUTDOWN

- 1. **Direct Dependents Hit:** SIL (Silicon Wafer) & SAND (Silica Sand)
- 2. **Secondary Cascade:** PCB (Circuit Boards), SEN (Sensors), GLS (Glass)
- 3. **Major Subsystems:** EL (Electronics) and BD (Body & Exterior)
- 4. **Final Assembly Impact:** APEX MFG HALTED

### Simulation Metrics:

- Blast Radius: 10 Suppliers Paralyzed
- Final Assembly Impact: HALTED (Tier 0)
- Secondary Backup Contracted: YES
- Strategy: Engage pre-contracted secondary mine immediately.

## Graph Visualization & Alert Engine

## VISUAL ENCODING PRINCIPLES

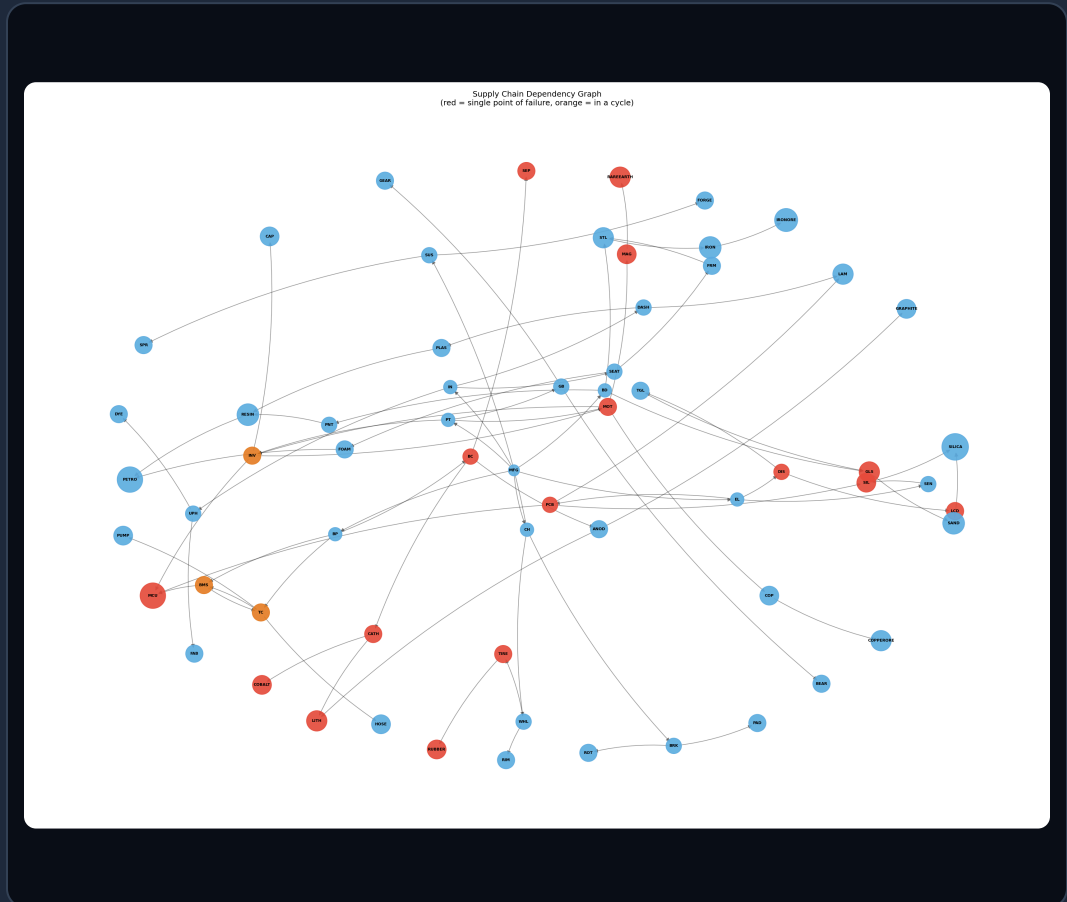
- **Color Coding Semantics**

- **RED:** Single Point of Failure (SPOF)
- **ORANGE:** Trapped in circular dependency
- **BLUE:** Standard stable supplier

- **Node Size Proportional to Blast Radius**

```
node_size = 300 + (score / max_score) * 1400
```

- **Automated Report Generation**



# Automated Unit Testing Suite (100% Pass Rate)

6 VERIFIED TEST CASES IN TESTS/TEST\_GRAPH\_ALGORITHMS.PY

✓ test\_graph\_loading() [PASSED]

Verifies 63 suppliers, 76 dependencies, Apex Motors root MFG tier 0, and metadata dictionary bindings.

✓ test\_spof\_detection() [PASSED]

Verifies all 16 SPOFs identified. Asserts BC and MOT are flagged while SILICA (backup=True) is exempted.

✓ test\_criticality() [PASSED]

Verifies reverse BFS predecessor reachability. Confirms SILICA >= 10 affected nodes and MCU >= 9 affected nodes.

✓ test\_cycle\_detection() [PASSED]

Verifies exactly 2 circular loops detected via 3-color DFS. Asserts membership of BMS, TC, MOT, and INV.

✓ test\_dijkstra() [PASSED]

Verifies minimum lead-time distance calculations from root MFG: MFG=0d, BP=10d, PT=12d, and RAREEARTH=85d max.

✓ test\_alternate\_path() [PASSED]

Verifies alternate route search. Confirms MFG can reach target BP when avoiding intermediate CH.

# Summary of Work Delivered & Roadmap

## ACCOMPLISHMENTS DELIVERED SO FAR

✓ **High-Performance Graph Engine**

Dual adjacency list  $O(V + E)$  memory layout with  $O(1)$  bi-directional neighbor and predecessor lookups.

✓ **Traversal & Sequencing Algorithms**

3-Color DFS cycle detection discovering 2 deadlocks; Kahn's topological sort build sequence validator.

✓ **Multi-Factor Risk Assessment**

Reverse BFS blast-radius scoring; 16-node SPOF audit; min-heap Dijkstra lead-time solver; alternate path router.

✓ **Verification & Visual Artifacts**

6 unit tests passing 100%; publication-ready Matplotlib dependency graph PNG output.

## FUTURE ALGORITHMIC ROADMAP

→ **Stochastic Lead-Time Variations**

Incorporate dynamic shipping delay variances and port congestion probabilities into Dijkstra calculations.

→ **Maximum Flow / Minimum Cut (Edmonds-Karp)**

Model component volume throughput capacities to identify delivery bottlenecks from Tier 4 mines to Tier 0 assembly.

→ **Safety Stock Buffer Optimization**

Algorithmically compute optimal inventory buffer sizing for the 16 identified Single Points of Failure.